

🎵 AI Data Analytics Agent

A natural-language agent that answers questions about a real multi-table database — **without being given the schema in advance**. It explores the database itself, writes and executes its own SQL (including multi-table joins), remembers context across follow-up questions, and shows its work at every step.

🔗 **Live demo:** <https://dataagent-yrtg26mzxaf4cnkhndfz4h.streamlit.app/>

Built as a prototype to demonstrate agentic, tool-using AI pipelines — the sample data here is a public music-store dataset (Chinook), used purely as an example domain. The same architecture applies to any relational database.

What it does

Ask a question like *"Which artist has generated the most revenue, and how many unique customers bought their music?"* and the agent will:

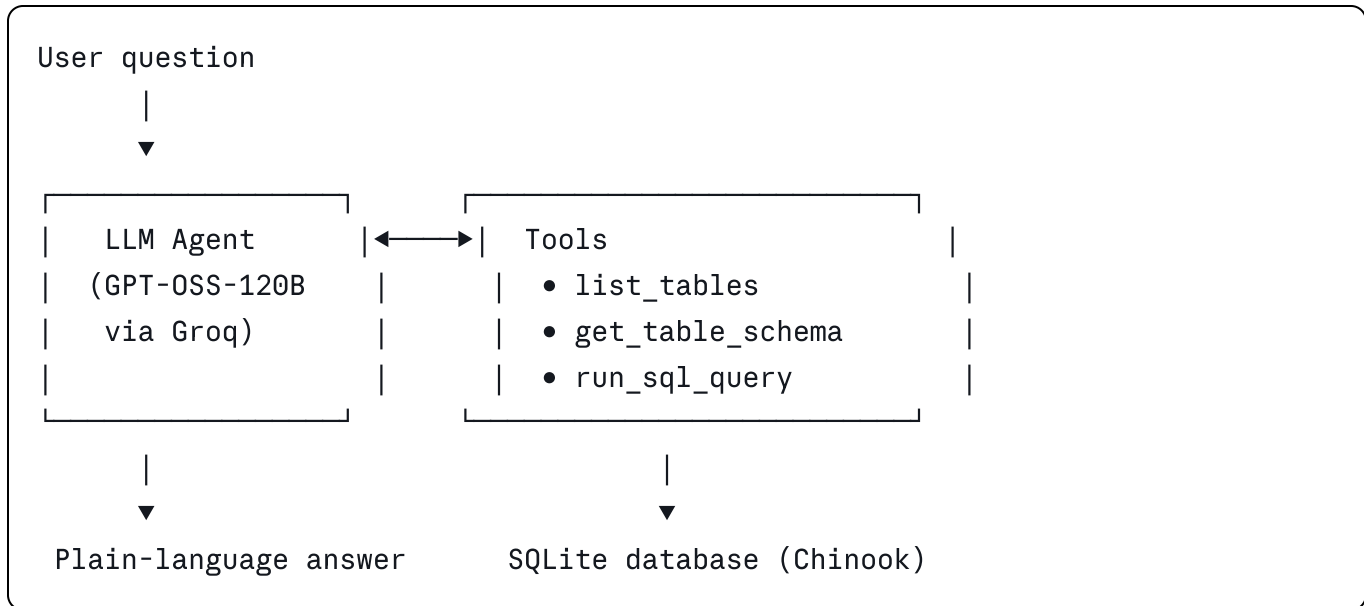
1. **Explore** — call `list_tables` and `get_table_schema` to discover the real database structure, since it isn't told the schema upfront
2. **Reason** — figure out how tables relate to each other (foreign keys, multi-hop joins) to find a path to the answer
3. **Query** — write and execute real SQL, including joins across up to 5 tables
4. **Answer** — explain the result in plain language, grounded in the actual returned data
5. **Remember** — reuse everything it learned for follow-up questions in the same conversation, without re-exploring from scratch

Every response shows its exploration steps, the exact SQL it ran, and the raw result table — full transparency, not a black box.

Example questions

- "Who are the top 5 customers by total spending?"
- "Which employee has generated the most sales?"
- "Which artist has generated the most revenue, and how many unique customers bought their music?"
- Follow-up: "Now break that down by country"

Architecture



The agent is given **only three tools** — it has no prior knowledge of table or column names. This mirrors how a real analyst approaches an unfamiliar database, and how production text-to-SQL agents are typically built for large/evolving schemas where dumping the full schema into every prompt doesn't scale.

Reliability safeguards

Getting LLM tool-use to behave reliably in practice required real debugging, not just a happy-path demo:

- **Repeat-call detection** — if the model tries to redo an identical exploration step, it's shown the cached result instead of wasting a round
- **Fake-response filtering** — catches cases where the model emits tool-call-like text instead of a real answer or a real tool call
- **Grounded-answer enforcement** — the model is explicitly instructed not to contradict its own query results, and raw data is always shown alongside the summary so it's independently verifiable
- **Error surfacing** — any internal failure is shown directly in the chat instead of failing silently

Tech stack

- **LLM:** GPT-OSS-120B via [Groq](#) (free tier)
- **Agent framework:** raw function/tool-calling (no LangChain — built from the API primitives directly)
- **Database:** SQLite (Chinook sample dataset, 11 tables)
- **UI:** Streamlit
- **Hosting:** Streamlit Community Cloud (free)

Running it locally

```
bash

# 1. Install dependencies
pip install -r requirements.txt

# 2. Get a free API key at https://console.groq.com
export GROQ_API_KEY="your-key-here"

# 3. Run
streamlit run streamlit_app.py
```

Files

File	Purpose
streamlit_app.py	Chat UI — exploration steps, SQL, and raw results all visible
data_agent.py	Core agent logic — tool definitions, exploration loop, safeguards
chinook.db	Sample database (music store, 11 tables)
setup_chinook.py	Script to re-download the sample database
requirements.txt	Python dependencies

Roadmap

- This is an intentional prototype. Planned next steps to move it toward production-readiness:
- ☐ Role-based access control / query-level permissions
 - ☐ Support for production databases (Postgres, Snowflake) instead of SQLite
 - ☐ Observability/logging for every agent action
 - ☐ Support for write-actions (not just read-only queries) behind explicit safeguards
 - ☐ Automated evaluation suite to measure query accuracy across a benchmark question set

Built by [Guntaka Sudhama](#) — [GitHub](#)